

EvoLabAsAService - Guia de Utilizacao para o Chatbot

Documento de conhecimento textual para orientar utilizadores durante a experimentacao com EvoLab.

1. Objetivo da aplicacao

O EvoLabAsAService e uma aplicacao web para criar, configurar e executar experiencias de evolucao de programas. O utilizador fornece um programa inicial, uma funcao de avaliacao e uma configuracao de modelo LLM. A plataforma usa OpenEvolve para gerar variacoes do programa, avaliar cada iteracao e guardar metricas, checkpoints e a melhor solucao encontrada.

O chatbot deve ajudar o utilizador a entender este fluxo, explicar conceitos da interface e orientar a experimentacao sem inventar funcionalidades que nao existem na aplicacao.

2. Fluxo recomendado para um novo utilizador

Quando um utilizador pergunta como comecar, o chatbot deve recomendar esta ordem:

1. Entrar na aplicacao atraves de Login, ou criar conta em Register caso ainda nao tenha conta.
2. Criar e validar uma credencial LLM em API Credentials.
3. Criar uma configuracao em Configuration Files, escolhendo a credencial validada, o modelo e os parametros evolutivos.
4. Criar um projeto em Projects, associando uma configuracao, um programa inicial e uma funcao de avaliacao.
5. Clicar em Start no projeto para colocar a experiencia na fila de execucao.
6. Acompanhar o estado do projeto e abrir Details para ver metricas, checkpoints e a solucao final.

A ordem e importante porque um projeto so consegue arrancar corretamente quando tem uma configuracao valida, codigo inicial e codigo de avaliacao. Se faltar algum destes elementos, a experiencia nao deve ser iniciada.

3. Autenticacao

Login

A pagina Login permite iniciar sessao com email e password ou com Google. O login local valida o formato do email e mostra erro quando as credenciais estao incorretas.

- Se o utilizador ja tem conta local, deve introduzir email e password e clicar em Sign in.
- Se preferir autenticacao externa, pode usar Sign in with Google.
- Apos login com sucesso, a aplicacao atualiza o estado global de autenticacao e redireciona para a pagina inicial.

Register

A pagina Register permite criar uma conta local com nome, email e password. Tambem existe opcao de registo com Google.

- O email deve ter formato valido.
- A password deve respeitar a validacao do backend; quando for demasiado fraca, a interface apresenta erro.
- Se o email ja existir, a aplicacao informa que a conta ja esta registada.

4. Credenciais LLM

As credenciais permitem que o EvoLab chame modelos externos ou locais durante a evolucao. Cada credencial pertence ao utilizador autenticado e e guardada no backend de forma encriptada.

Fornecedores suportados

- OpenAI: exige uma API key com formato esperado para OpenAI.
- Google Gemini: exige uma API key Gemini, normalmente iniciada por AIza.

- Local Model: permite usar um modelo local compatível com API tipo OpenAI, indicando porta e nome do modelo.

Como criar uma credencial

1. Abrir a página API Credentials.
2. Clicar em New Credential.
3. Escolher o provider: OpenAI, Google Gemini ou Local Model.
4. Introduzir a API key. Para Local Model, indicar também Model Name e Port.
5. Clicar em Create.
6. Usar Validate para confirmar que a credencial está operacional.

Validação e erros comuns

- Uma credencial inválida impede execuções bem sucedidas; o projeto pode falhar se a API key estiver errada.
- OpenAI e Gemini validam a chave chamando os endpoints de modelos dos respectivos fornecedores.
- Local Model valida o endpoint local na porta configurada e, se indicado, confirma que o modelo existe.
- Uma credencial associada a configurações não deve ser apagada sem remover primeiro as configurações/projetos dependentes.
- A aplicação assume uma credencial por provider para cada utilizador; para trocar a chave, deve atualizar a credencial existente.

5. Configurações

Uma configuração define como o OpenEvolve vai executar a experiência: que credencial e modelo usar, quantas iterações fazer, de quanto em quanto tempo guardar checkpoints e que parâmetros avançados aplicar.

Como criar uma configuração

1. Abrir Configuration Files.
2. Clicar em New Config.
3. Selecionar uma LLM Credential válida.
4. Escolher o Model Name adequado ao provider. Modelos Gemini devem usar credenciais Gemini; modelos OpenAI devem usar credenciais OpenAI.
5. Definir Max Iterations.
6. Definir Checkpoint Interval.
7. Opcionalmente abrir Advanced Parameters e ajustar parâmetros específicos.
8. Clicar em Save Configuration.

Regras importantes

- A credencial e a ligação a projeto são irreversíveis depois de criada a configuração; para alterar esses elementos, deve criar uma nova configuração.
- O nome do modelo não pode estar vazio.
- Max Iterations deve ser maior do que zero.
- Checkpoint Interval deve ser maior do que zero e não pode ser superior a Max Iterations.
- Se o modelo parecer Gemini mas a credencial for OpenAI, ou vice-versa, a configuração é considerada incoerente.
- Para modelos locais, a aplicação define automaticamente llm.api_base para apontar para host.docker.internal e a porta configurada, caso o utilizador não forneça outro valor.

6. Parametros avancados

Os parametros avancados devem ser explicados como controlos opcionais. O utilizador pode deixar os campos vazios para usar os valores por defeito definidos pelo backend.

LLM

- Temperature: controla variabilidade/criatividade. Valores baixos tornam as respostas mais previsiveis; valores altos aumentam exploracao mas tambem risco de instabilidade.
- Top P: controla amostragem por probabilidade cumulativa. Valores proximos de 1 permitem mais diversidade.
- Max Tokens: limita o tamanho maximo da resposta do modelo.
- Timeout: tempo maximo de espera por uma resposta do modelo.
- Retries: numero de tentativas em caso de falha temporaria.

Prompt

- System Message: instrucao principal que orienta o comportamento do modelo durante a evolucao.
- Num Top Programs: quantos dos melhores programas anteriores sao usados como referencia.
- Num Diverse Programs: quantos programas diferentes/diversos sao incluidos para evitar convergencia prematura.
- Include Artifacts: controla se artefactos adicionais podem ser incluidos no contexto da evolucao.

Database evolutiva

- Population Size: numero de programas mantidos na populacao ativa.
- Archive Size: tamanho do arquivo historico de solucoes.
- Num Islands: numero de subpopulacoes evolutivas independentes.
- Migration Interval: frequencia com que solucoes migram entre ilhas.
- Migration Rate: proporcao de solucoes migradas.
- Elite Selection Ratio: proporcao reservada para melhores solucoes.
- Exploration Ratio: peso dado a explorar solucoes novas.
- Exploitation Ratio: peso dado a melhorar solucoes promissoras.
- Feature Dimensions: dimensoes usadas para caracterizar diversidade, por defeito complexity e diversity.
- Feature Bins: discretizacao usada nas dimensoes de diversidade.

Evaluator

- Timeout: tempo maximo para avaliar uma solucao.
- Max Retries: repeticoes quando a avaliacao falha.
- Parallel Evaluations: numero de avaliacoes em paralelo.
- Cascade Evaluation: permite uma avaliacao por etapas, usando thresholds progressivos.
- Cascade Threshold 1, 2 e 3: limites crescentes usados quando a avaliacao em cascata esta ativa.

General

- Diff-Based Evolution: quando ativo, a evolucao pode trabalhar com diferencas/patches em vez de reescrever sempre o programa completo.

7. Projetos

Um projeto representa uma experiencia concreta. Ele junta o objetivo da experiencia, uma configuracao, um programa inicial e a logica que avalia se as solucoes geradas sao boas.

Como criar um projeto

1. Abrir Projects.

2. Clicar em New Project.
3. Introduzir Name. O nome é obrigatório e deve ser único por utilizador.
4. Adicionar Description se ajudar a identificar o objetivo.
5. Selecionar uma Configuration existente ou deixar sem configuração para atribuir depois.
6. Introduzir ou carregar o Initial Program Code.
7. Introduzir ou carregar a Evaluation Function / Fitness Logic.
8. Clicar em Save Project.

Programa inicial

O Initial Program Code é o ponto de partida da evolução. Deve conter uma implementação inicial funcional ou pelo menos uma base que o OpenEvolve possa modificar. Quanto mais claro for o código inicial, mais fácil será avaliar melhorias.

Função de avaliação

A Evaluation Function / Fitness Logic define como cada solução é pontuada. Esta é uma das partes mais importantes: se a avaliação não mede corretamente o objetivo, o sistema pode otimizar o comportamento errado.

- Deve devolver ou produzir uma métrica de fitness comparável entre iterações.
- Deve ser determinística sempre que possível.
- Deve falhar de forma clara quando o programa gerado é inválido.
- Deve ser suficientemente rápida para permitir múltiplas iterações.

8. Arrancar uma experiência

Para clicar em Start, o projeto deve estar no estado CREATED e estar pronto para executar. Um projeto pronto tem configuração associada, programa inicial e avaliador.

Estados do projeto

- CREATED: projeto criado e ainda editável.
- QUEUED: projeto colocado na fila de execução.
- RUNNING: um worker está a executar o OpenEvolve para esse projeto.
- COMPLETED: a execução terminou com sucesso.
- FAILED: a execução falhou.

Projetos em RUNNING ou QUEUED não devem ser editados. Projetos COMPLETED ou FAILED podem ser reiniciados através de Restart, voltando ao estado CREATED.

O que acontece internamente

- O backend coloca o projeto numa fila de execução.
- Um dos workers disponíveis retira o projeto da fila.
- O backend cria um job e gera um ficheiro YAML temporário de configuração OpenEvolve.
- É iniciado um container Docker com a imagem ghcr.io/algorithmicsuperintelligence/openevolve:latest.
- O container recebe initial_program.py, evaluator.py e config.yaml.
- No fim, o backend recolhe logs, melhor solução, métricas de iteração e checkpoints.

9. Details, métricas, checkpoints e resultado final

Details

A página Details de um projeto mostra o histórico da execução mais recente. Se ainda não houver jobs, a aplicação informa que o projeto ainda não tem histórico e deve ser iniciado na página Projects.

Mettricas

- Fitness Score: pontuacao da solucao numa iteracao. Valores mais altos representam melhor desempenho segundo o avaliador.
- Execution Time: tempo gasto na iteracao, quando disponivel.
- O grafico mostra a evolucao do fitness ao longo das iteracoes.

Checkpoints

Checkpoints sao solucoes guardadas ao longo da evolucao, de acordo com o Checkpoint Interval. Permitem ver codigo intermediario e perceber como a solucao foi melhorando.

Final Result

Quando existe bestSolution, a interface apresenta Final Result e permite abrir View Final Solution. Esta e a melhor solucao encontrada pelo OpenEvolve naquela execucao.

10. Problemas frequentes e respostas esperadas do chatbot

Nao consigo iniciar o projeto

- Confirmar se o projeto esta em CREATED.
- Confirmar se tem Configuration associada.
- Confirmar se Initial Program Code nao esta vazio.
- Confirmar se Evaluation Function / Fitness Logic nao esta vazia.
- Confirmar se a credencial associada a configuracao esta valida.

A credencial aparece invalida

- Verificar se a API key foi copiada corretamente e sem espacos.
- Verificar se o provider escolhido corresponde a chave: OpenAI para chaves OpenAI, Gemini para chaves Gemini.
- Para Local Model, verificar se o servidor local esta ativo, se a porta esta correta e se o modelo existe.

Nao consigo apagar uma credencial

Se a credencial estiver ligada a uma configuracao, a aplicacao pode bloquear a remocao. O utilizador deve remover ou recriar configuracoes/projetos dependentes antes de apagar a credencial.

A configuracao da erro

- Verificar se Max Iterations e maior que zero.
- Verificar se Checkpoint Interval e maior que zero e menor ou igual a Max Iterations.
- Verificar se o modelo corresponde ao provider da credencial.
- Nao colocar API keys nos parametros avancados; as chaves pertencem ao modulo de credenciais.

O projeto falhou

- Verificar se a credencial continua valida.
- Verificar se o programa inicial tem sintaxe valida.
- Verificar se a funcao de avaliacao executa corretamente.
- Verificar se o timeout do avaliador e suficiente.
- Verificar se o modelo escolhido existe e esta disponivel para a API key.

11. Comportamento esperado do chatbot

- Responder em linguagem clara e orientada a passos praticos.
- Quando o utilizador estiver perdido, recomendar o fluxo: login, credencial, configuracao, projeto, start, details.

- Explicar conceitos como credential, config, project, max iterations, checkpoint interval, temperature, evaluator e fitness score.
- Distinguir entre modelo de resposta LLM e modelo de embeddings quando a pergunta for sobre o chatbot.
- Não prometer funcionalidades não implementadas na aplicação.
- Quando a pergunta envolver erro de execução, pedir ao utilizador para confirmar estado do projeto, validade da credencial, código inicial, avaliador e configuração.
- Quando houver risco de apagar dados ou recriar configurações, avisar o utilizador antes de recomendar a ação.
- Se não houver informação suficiente, pedir o detalhe mínimo necessário em vez de inventar uma causa.

12. Resumo curto para onboarding

Para usar o EvoLab: cria conta ou faz login, cria uma credencial LLM e valida-a, cria uma configuração com essa credencial e modelo, cria um projeto com código inicial e avaliador, inicia a experiência e acompanha os resultados em Details. A qualidade da experiência depende sobretudo de três elementos: uma credencial válida, uma configuração coerente e uma função de avaliação que realmente mede o objetivo pretendido.